



KAUNAS UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATICS
DEPARTMENT OF MULTIMEDIA ENGINEERING
Software Engineering Course Team Project

DormLife

A web application for students living in dormitories

Project Report

Team "D-opps"

Supervisor

Prof. C. Canbulut

Role	Team member
Product Owner / Scrum Master	Nika Merabishvili
Requirements Architect	Polina Sevastianova
Developer	Denys Suntsov
Developer	Vladyslava Yaroshenko
Tester	Nuka Mshvenieradze

KAUNAS, 2026

SUMMARY

This report documents the design, development, and delivery of DormLife, a full-stack web application built by a five-person student team during the Software Engineering course at Kaunas University of Technology. The product is targeted at students who live in dormitories and bundles several everyday needs — a peer-to-peer marketplace, a laundry-machine availability tracker, a multi-channel chat system, a dormitory events module, and an in-app notification center — into a single Single Page Application.

The team applied an Agile process based on Scrum throughout the semester. Work was organized into time-boxed sprints, tracked in Jira under the DOSK2 project, and developed against a Git workflow with feature branches, pull requests, and a GitHub Actions continuous-integration pipeline that automatically runs the backend test suite on every push to the main branch.

The system is implemented as a React 18 and TypeScript front end backed by a Django REST Framework API with JWT and Google OAuth 2.0 authentication. Data is stored in SQLite during local development and in PostgreSQL 15 in staging, which is orchestrated with Docker Compose. By the end of the final sprint (Sprint 6), all planned core features have been implemented end-to-end, the application has been validated through manual and automated testing, and the system has been deployed to a staging environment for the final demonstration.

TABLE OF CONTENTS

SUMMARY	2
GLOSSARY OF TERMS AND ABBREVIATIONS	4
1. INTRODUCTION	5
1.1. Purpose of the document.....	5
1.2. Aim	5
1.3. Work Objective	5
2. ANALYSIS OF THE PROBLEM DOMAIN AND CHOSEN SOLUTION	6
2.1. Problem domain analysis	6
2.2. Analysis of the technology stack.....	6
2.2.1. Front end — React, TypeScript, and Tailwind CSS.....	6
2.2.2. Back end — Django and Django REST Framework.....	6
2.2.3. Database and infrastructure	7
2.3. Analysis of existing systems	7
3. SYSTEM DESIGN	8
3.1. System functionality	8
3.1.1. Functional requirements	8
3.1.2. Non-functional requirements.....	9
3.1.3. Process model	10
3.1.4. System architecture and deployment	10
3.2. Activity of the software	11
3.3. Class diagram of the system	13
3.4. Domain model overview.....	14
3.5. REST API surface	15
4. PROJECT MANAGEMENT WITH SCRUM	16
4.1. Team roles	16
4.2. Tools.....	16
4.3. Branching strategy	16
4.4. Sprint history (summary)	16
5. TESTING	18
5.1. Testing approach	18
5.2. Continuous integration pipeline	18
5.3. Automated tests	18
5.4. Manual end-to-end test cases.....	19
5.5. Bugs found and resolved.....	19
6. CONCLUSION	21
REFERENCES	22

GLOSSARY OF TERMS AND ABBREVIATIONS

Term	Meaning
SPA	Single Page Application — a web application that loads a single HTML page and dynamically updates content via JavaScript without full page reloads.
SCRUM	An iterative, incremental Agile framework for managing product development, structured around fixed-length sprints.
MVP	Minimum Viable Product — the smallest version of a product that delivers value and can be released to users.
JIRA	An issue-tracking and project-management tool used by the team to plan sprints, manage the backlog, and track work items.
JWT	JSON Web Token — a compact, signed token used to authenticate API requests.
OAuth 2.0	An open standard for delegated authorization; used here for Google sign-in.
REST	Representational State Transfer — an architectural style for stateless web APIs.
DRF	Django REST Framework — a library that adds REST API tooling on top of Django.
ORM	Object-Relational Mapping — Django's mechanism for mapping Python classes to database tables.
CI/CD	Continuous Integration / Continuous Delivery — automated build, test, and deployment pipelines.
UUID	Universally Unique Identifier — used as the primary key for user records.
RSVP	Répondez s'il vous plaît — a user's confirmation of attendance to an event.

1. INTRODUCTION

Dormitory life concentrates hundreds of students into a single building or campus, yet the everyday tools available to them are surprisingly fragmented. Students looking to sell or exchange used items rely on improvised messaging-app threads. Information about which laundry machine is free is communicated by walking down to the laundry room and looking. Announcements about social events spread by word of mouth or via posters in the corridor. The motivating idea for this project — proposed by a team member who is an Erasmus student living in a dormitory — is that a single web application tailored to dormitory life can replace several of these ad-hoc workarounds at once.

DormLife is the team's response to that problem. It is a Single Page Application that brings together a marketplace for second-hand items, a laundry-machine availability tracker, a multi-level chat system, a dormitory events module, and an in-app notification center. The application was developed over one academic semester by a team of five students, each in a clearly defined Scrum role, using the practices and tools the course is designed to teach.

1.1. Purpose of the document

This document is the final project report submitted as the deliverable of the Software Engineering course. Its purpose is to describe what the team built, why specific design and technology decisions were taken, how the work was organized using Scrum and Jira, and how the resulting system was validated through testing. The report is intended for the course supervisor and serves both as a summary of the semester's work and as a self-contained reference for the DormLife system.

1.2. Aim

The aim of the project is to deliver a working, deployable web application — DormLife — that helps students living in dormitories connect with each other, share or exchange resources, and access information that is useful in everyday dormitory life. By the end of the semester the system covers all core features in a state that can be demonstrated to the course supervisor.

1.3. Work Objective

The work objectives of the project are:

- To analyze the everyday needs of dormitory students and translate them into a concrete set of product features.
- To select and justify a modern web technology stack suitable for a small team and a single-semester schedule.
- To design the system's data model, REST API, and front-end architecture.
- To implement the agreed feature set incrementally across multiple Scrum sprints, with all work tracked in Jira.
- To apply professional software-engineering practices including Git branching, code review via pull requests, and a GitHub Actions continuous-integration pipeline.
- To validate the system through a combination of manual end-to-end testing and automated tests.
- To document the resulting system, including known issues and ideas for future work.

2. ANALYSIS OF THE PROBLEM DOMAIN AND CHOSEN SOLUTION

Before any code was written, the team analyzed the problem domain, the technology landscape, and existing comparable products. This analysis informed both the feature set and the technology stack.

2.1. Problem domain analysis

Dormitory students share a set of recurring everyday needs that are poorly served by general-purpose tools. The team identified four needs as the most concrete and the most amenable to a software solution:

- Buying, selling, and exchanging second-hand items — especially relevant when students arrive at the start of a semester or move out at the end.
- Checking whether laundry machines are free without physically walking to the laundry room.
- Communicating with other students in the same dormitory, in other dormitories, or in a shared global channel.
- Discovering and joining events organized inside the dormitory.

In addition, two cross-cutting needs emerged: students should be notified when something happens that concerns them (a new message, an exchange offer, a new event), and they should be able to maintain a profile that links them to their dormitory and room.

2.2. Analysis of the technology stack

The team chose a modern, mainstream stack that is widely used in industry, well-documented, and feasible to learn and apply within one semester.

2.2.1. Front end — React, TypeScript, and Tailwind CSS

The front end is built with React 18, TypeScript, and the Vite build tool. React is the most widely used component-based UI library and has strong learning resources, which lowered the onboarding cost for the developers. TypeScript adds static typing, which catches many integration errors at build time rather than at runtime. Vite provides a fast development server and an optimized production build.

For styling, the team chose Tailwind CSS v4 together with the shadcn/ui component library, which is built on top of the Radix UI primitives. This combination provides accessible, unstyled component foundations (dialogs, menus, popovers, etc.) that the team can theme consistently with utility classes, instead of writing a large amount of custom CSS from scratch.

2.2.2. Back end — Django and Django REST Framework

The back end uses Python with Django and the Django REST Framework (DRF). Django provides a mature ORM, a built-in admin interface that the team uses for inspecting data during development, and a strong ecosystem of well-maintained packages — including `django-rest-framework-simplejwt` for JWT authentication, `django-rest-passwordreset` for the password-reset flow, and `django-cors-headers` for handling cross-origin requests from the front end.

Authentication combines email and password (JWT-based) with Google OAuth 2.0 through the `@react-oauth/google` library, so that students can sign in with whichever method is more convenient.

2.2.3. Database and infrastructure

SQLite is used as the local development database because it requires no setup and ships with Python. For the staging environment that is used for the final demonstration, the team uses PostgreSQL 15 orchestrated with Docker Compose, alongside the Django back end. This mirrors a realistic deployment configuration without forcing every developer to install PostgreSQL locally.

2.3. Analysis of existing systems

Several general-purpose tools already cover parts of the problem domain. Marketplace functionality is available on platforms such as Facebook Marketplace and Vinted. Messaging is dominated by WhatsApp, Telegram, and Messenger. Event discovery is fragmented across Facebook Events, Instagram, and corridor posters.

None of these tools, however, are organized around the dormitory as a unit. They do not know which dormitory a user belongs to, they do not provide a dormitory-scoped chat channel, they do not show whether the laundry machine on the user's floor is free, and they do not group exchange offers by dormitory. DormLife's value is not in re-inventing chat or marketplaces, but in centering all of these features on a single concept — the dormitory community — and removing the need for students to glue several unrelated tools together.

The comparison can be summarized as follows:

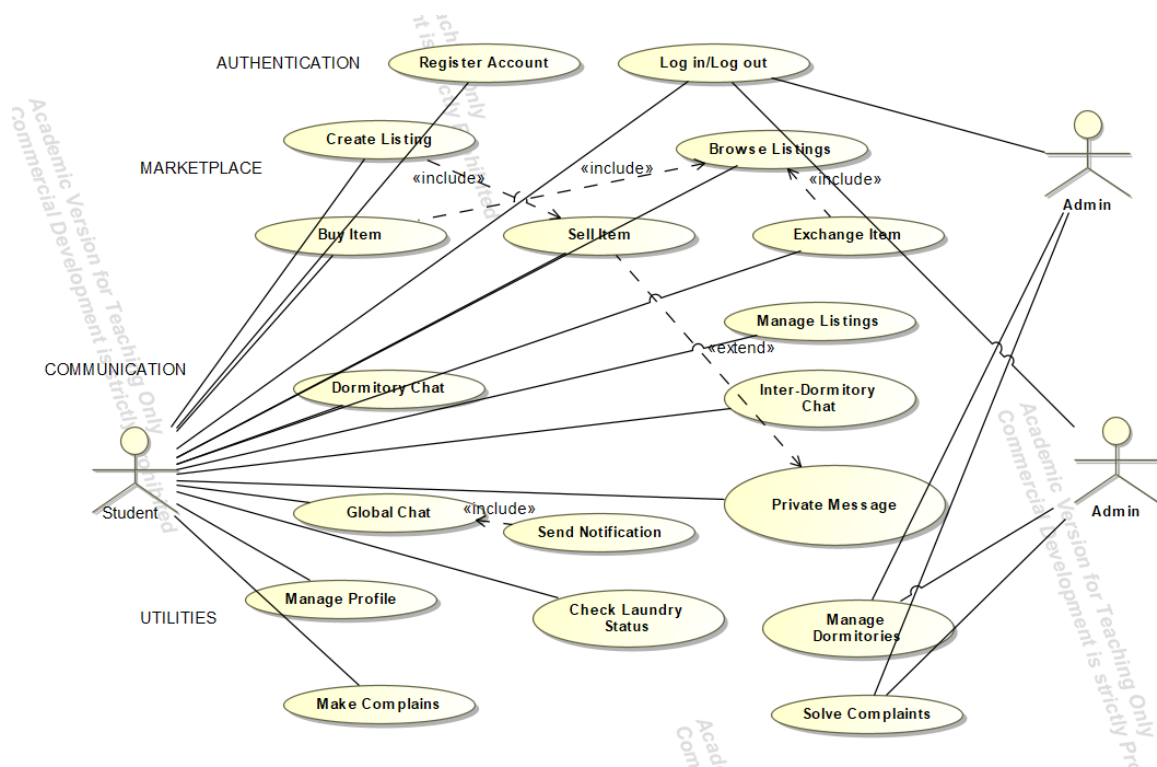
Feature	General tools	Dormitory bulletin board	DormLife
Marketplace	Yes (Facebook, Vinted)	Limited / paper	Yes, dormitory-aware
Laundry availability	No	No	Yes, with auto-release timer
Dormitory chat	Improvised groups	No	Yes, scoped per dormitory + global
Events	Yes, general	Paper posters	Yes, dormitory-scoped
Single integrated app	No	No	Yes

Table 1. Comparison of DormLife with existing alternatives

3. SYSTEM DESIGN

3.1. System functionality

The DormLife system functionality is split between the authenticated end-user features (students, doorkeepers, administrators) and the cross-cutting concerns (authentication, notifications, profile).



3.1.1. Functional requirements

The following functional requirements are implemented in the final build:

Authentication and accounts

- The system shall allow a new user to register with email, password, full name, role (student / doorkeeper / admin), dormitory number, and room number.
- The system shall allow registered users to log in with email and password, returning a JWT access token (one-day lifetime) and a refresh token (seven-day lifetime).
- The system shall allow users to log in using Google OAuth 2.0.
- The system shall provide a password-reset flow initiated by email.
- The system shall allow each user to view and edit their own profile, including their dormitory, room number, and profile photo.

Marketplace

- The system shall allow any authenticated user to create a listing with a title, price, description, category, primary photo, optional additional photos, and a flag indicating whether the item is used.
- The system shall allow users to browse and search listings by title and filter them by category and favorite status.
- The system shall allow users to favorite and unfavorite listings.

- The system shall allow a buyer to contact a seller, which opens a marketplace-scoped chat conversation linked to the product.
- The system shall allow a buyer to send the seller a structured exchange offer specifying what they want to trade.

Laundry tracker

- The system shall display all laundry machines (washers and dryers) located in the user's dormitory, each with a status of free, occupied, or out-of-order.
- The system shall display the remaining minutes for any occupied machine.
- The system shall allow users to update a machine's status.
- The system shall automatically release a machine back to the free state once its end time has passed.
- The system shall allow administrators to create and delete machines.

Chat

- The system shall support private one-to-one conversations between two users.
- The system shall support a product-scoped chat between a buyer and a seller.
- The system shall support a dormitory group chat that all members of the same dormitory share.
- The system shall support a global chat that all users share.
- The system shall mark messages as read when their conversation is opened.

Events

- The system shall allow users to create events with a title, description, date, and location, scoped to the creator's dormitory.
- The system shall allow users to RSVP or cancel their RSVP for an event.
- The system shall allow the creator or an administrator to delete an event.

Notifications

- The system shall create an in-app notification when a user receives a new message, a new exchange offer, or when a new event is created in the system.
- The system shall send an email notification for new messages.
- The system shall allow users to filter notifications by status (all/unread) and by type.
- The system shall allow users to mark a single notification or all notifications as read.

3.1.2. Non-functional requirements

- Usability: the system shall present a consistent visual style across pages, with a sidebar navigation that adapts to mobile (icon-only) and desktop (full text) viewports.
- Security: all API endpoints that expose user data shall require a valid JWT access token; passwords shall be stored hashed; OAuth tokens shall be validated against the Google userinfo endpoint.
- Performance: the front end shall fetch chat messages no less than once every three seconds and laundry-machine status no less than once every fifteen seconds, while keeping the user interface responsive.

- **Maintainability:** the code base shall be organized into a clear backend / frontend split, with database access encapsulated in Django models and front-end pages structured as React components.
- **Portability:** the staging environment shall run inside Docker Compose so that it can be reproduced on any developer machine.
- **Reliability:** every push to the main branch shall trigger the back-end test suite via GitHub Actions.

Performance Requirements

- The application shall run in any modern web browser (Chrome, Firefox, Safari, Edge) released within the last two years.
- The back-end API shall respond to standard list and detail requests within 500 ms under typical single-user load.
- The front end shall poll for new chat messages every 3 seconds and for laundry-machine status every 15 seconds without degrading the user interface responsiveness.
- The system shall support at least 50 concurrent authenticated users in the staging environment without requiring horizontal scaling.

Design Constraints

The application is built as a decoupled client–server system. The front end is a React 18 Single Page Application scaffolded with Vite and styled with Tailwind CSS v4 and the shadcn/ui component library. The back end is a Python application built on Django and Django REST Framework, exposing a stateless REST API authenticated with JWT tokens. During local development the database is SQLite; for the staging environment used in the final demonstration the database is PostgreSQL 15, orchestrated together with the Django back end via Docker Compose.

These technology choices were constrained by three factors: the team’s prior experience (all members had some exposure to Python and JavaScript), the single-semester timeline (which favoured mature frameworks with good documentation over newer alternatives), and the course requirement to demonstrate a working deployment (which led to the Docker-based staging setup rather than a cloud-hosted solution).

3.1.3. Process model

The team followed Scrum as taught in the course. Work was organized into time-boxed sprints, with a sprint planning session at the start, daily standups during the sprint, and a sprint review at the end. All product backlog items were captured as Jira tickets under the DOSK2 project, with story points used to estimate effort.

Branches were created per ticket using the naming convention DOSK2-{ticket}-{short-description}, in line with the team's documented branching strategy. Each branch ended with a pull request to main, and a feature was only considered done after code review by another team member and after the GitHub Actions pipeline reported a passing build. This linear history is preserved as a record of how each feature was developed.

3.1.4. System architecture and deployment

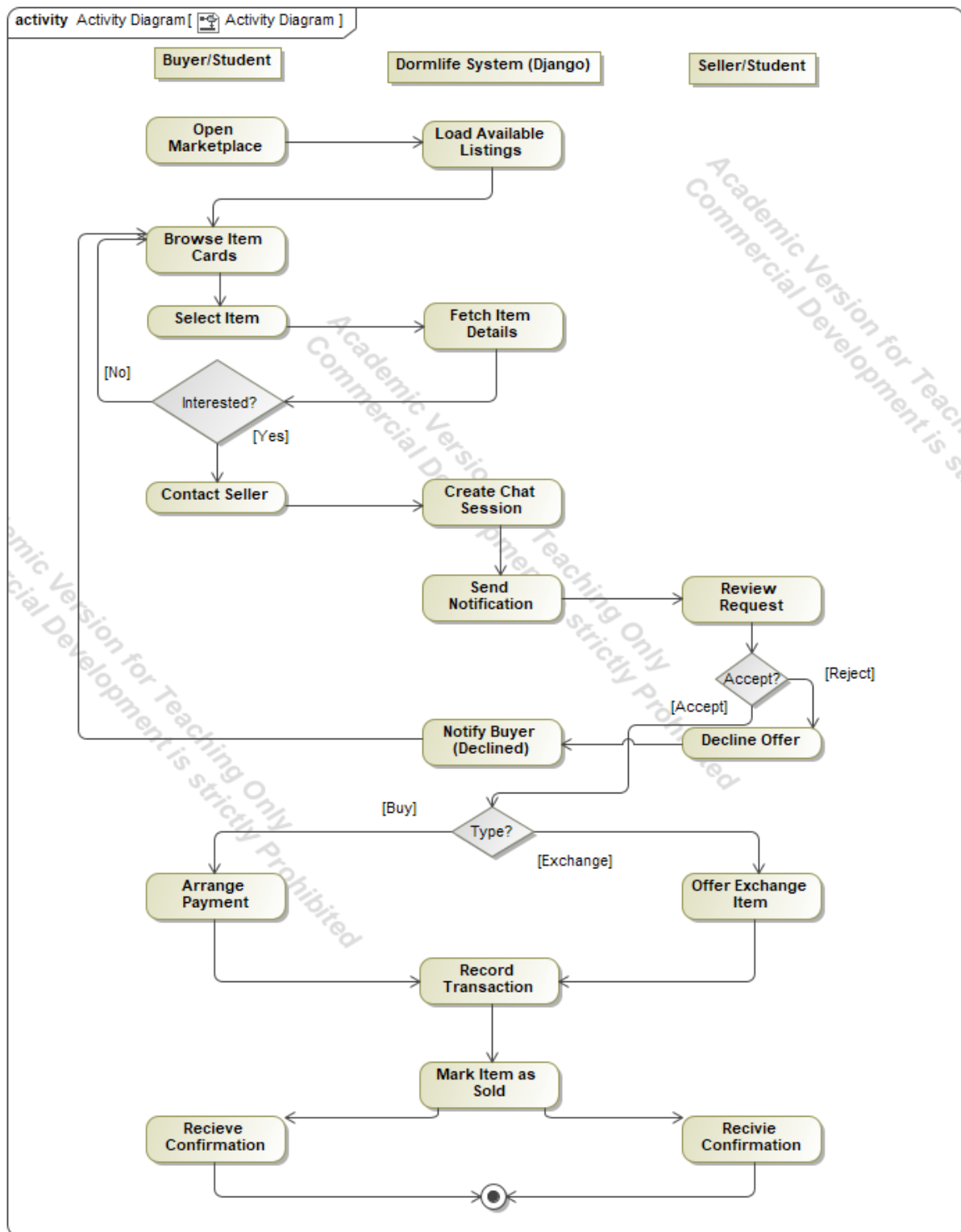
DormLife follows a classic two-tier client–server architecture. The React front end communicates with the Django back end exclusively through HTTP REST calls made with Axios, authenticated with a Bearer JWT token stored in browser localStorage. Static media (profile photos and product images) is served by Django from the local media directory in development; in staging it is mounted from a Docker volume.

Source layout: the front end lives in `proj/`, the back end in `backend/`, and the staging orchestration is defined in `docker-compose.staging.yml` at the repository root. The backend image is built from `backend/Dockerfile` and exposed on port 8001, while PostgreSQL 15 is exposed on port 5433. Configuration values that differ between environments (database credentials, Django SECRET_KEY, email password) are supplied through a `.env.staging` file that is intentionally not committed to the repository.

On the back end, all REST endpoints are grouped under `/api/` and routed through Django's URL configuration to a small number of DRF ViewSets and APIViews. The core domain model contains ten tables — User, Product, ProductPhoto, Machine, Conversation, Message, ExchangeOffer, Favorite, Event, and Notification — with foreign-key relationships that mirror the feature set. Notifications and email alerts are produced from Django signal handlers attached to the Message, ExchangeOffer, and Event models, which keeps the side effects out of the view layer.

3.2. Activity of the software

The activity diagram below models the marketplace transaction flow — the most interaction-heavy feature in DormLife. The diagram uses three swimlanes to separate responsibilities: the Buyer/Student who initiates the purchase or exchange, the DormLife System (Django) which orchestrates the back-end logic, and the Seller/Student who reviews incoming requests.

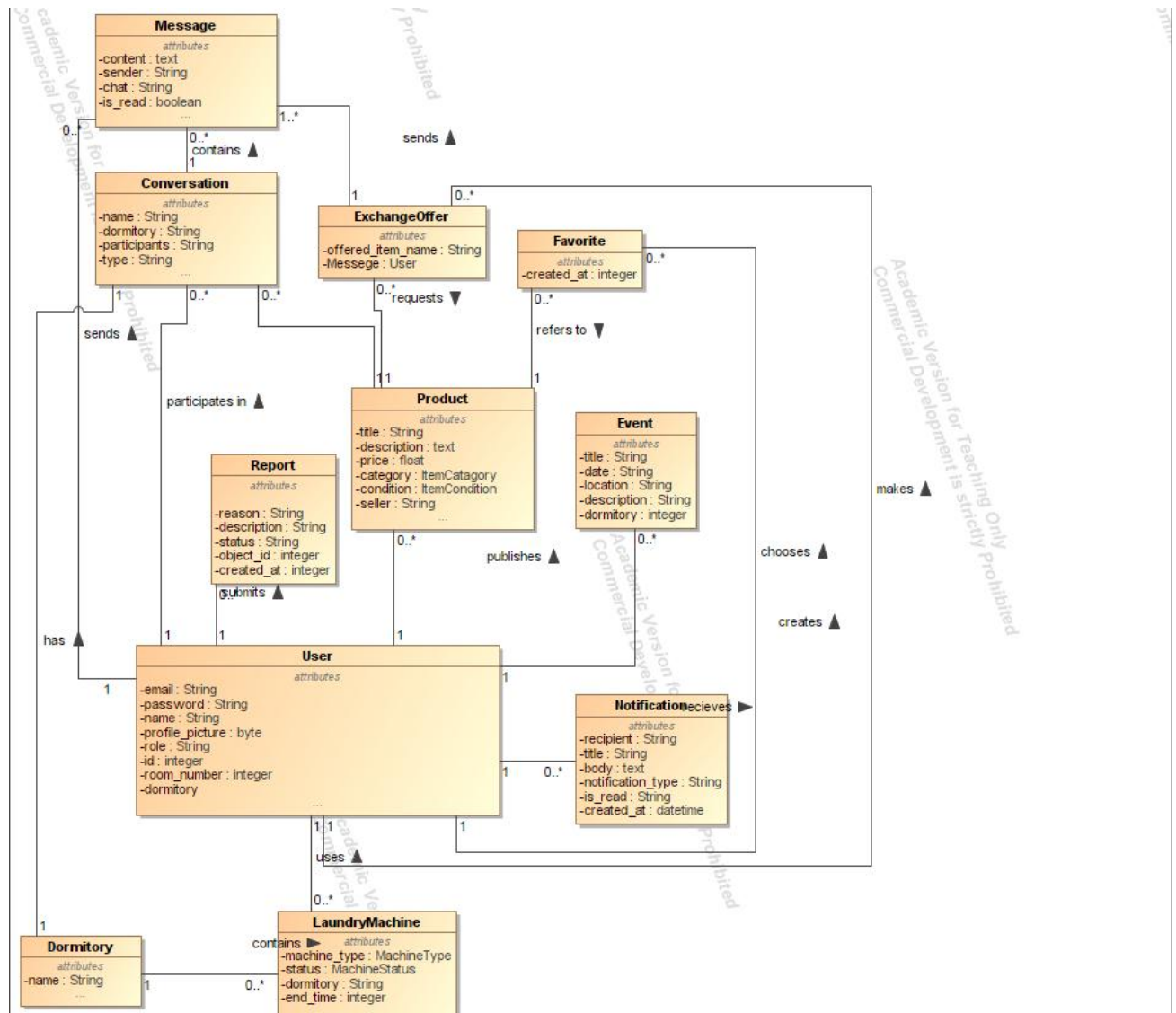


The flow begins when the buyer opens the marketplace page and the system loads all available listings. The buyer browses the item cards, selects one of interest, and the system fetches the full item details. A decision node checks whether the buyer is interested: if not, control loops back to browsing; if yes, the buyer contacts the seller and the system creates a chat session. A notification is sent to the seller, who reviews the request. A second decision node models the seller's response: if the seller rejects, the offer is declined and the buyer is notified; if the seller accepts, a third decision node determines the transaction type. For a purchase, the parties arrange payment; for an exchange, the buyer offers an

exchange item. Both paths converge into a shared sequence — record the transaction, mark the item as sold — after which both the buyer and the seller receive a confirmation before the flow reaches its final state.

3.3. Class diagram of the system

The class diagram below represents the full DormLife domain model. It follows UML conventions: classes show their attributes, data types, and operations; associations carry multiplicity labels and role names; and enumeration types are displayed separately on the right-hand side of the diagram.



The central class is **User**, which holds authentication credentials, a profile picture, a role, and an active flag, and exposes `register()`, `login()`, and `update_profile()` operations. Two specializations inherit from **User**: **Student** adds university, Erasmus status, dormitory, room number, and phone number, and provides domain operations such as `list_item()`, `buy_item()`, `exchange_item()`, `join_chat()`, `check_laundry()`, and `get_notifications()`; **Administrator** adds an admin level and a set of managed dormitories, with operations for moderating content, banning users, and viewing reports.

A Dormitory holds its name, address, geographic coordinates, total rooms, and an image, and exposes `get_residents()`, `get_chat_room()`, and `get_laundry_machines()`. Each Dormitory houses one or more Students and contains zero or more LaundryMachine instances, whose status is governed by the MachineType (washer/drier) and MachineStatus (free/in_use/out_of_order) enumerations.

Item represents a marketplace listing with title, description, price, category (ItemCategory), condition (ItemCondition), status (ItemStatus), images, and seller reference. Items are listed by a Student and linked to a Dormitory through an "is in" association. A Transaction records the agreed deal between a buyer and a seller: it carries a transaction type (TransactionType: purchase/exchange/free_giveaway), a status (TransactionStatus: pending/accepted/completed/cancelled/declined), an agreed price, and timestamps. Each Transaction includes one Item and optionally refers to a Review, which captures a rating, a comment, and the identities of the reviewer and the reviewed user.

ChatRoom supports five room types defined by the ChatRoomType enumeration — dormitory, inter_dormitory, global, private, and transaction — and tracks its participants and active state. Each ChatRoom receives zero or more Message objects, which store content, sender, timestamps, and a read flag. Finally, Notification delivers in-app alerts to users, and MapLocation stores points of interest with coordinates, categories, opening hours, and a `get_directions_url()` helper. A User receives zero or more Notifications, closing the model's event loop.

3.4. Domain model overview

The system manages ten persistent entities. Their responsibilities and relationships are summarized below.

Entity	Responsibility
User	Custom Django user identified by a UUID and an email address, extended with role, dormitory number, room number, and profile photo.
Product	A marketplace listing with title, price, category, description, primary photo, status (available / sold / reserved), and seller.
ProductPhoto	Zero-or-more additional images attached to a Product.
Machine	A washer or dryer in a specific dormitory, with status, location, an end time when occupied, and an automatic release rule.
Conversation	A chat thread of one of five types: private, market, group (dormitory), admin, or global.
Message	A single text message inside a conversation, with a sender, a read flag, and a creation timestamp.
ExchangeOffer	A structured trade offer from one user to another for a target Product.
Favorite	A unique pair (user, product) used to power the favorites list.
Event	A dormitory-scoped event with a date, location, creator, and attendees.
Notification	An in-app notification of type message, offer, event, or system, linked to its target object by id.

Table 2. Domain entities and their responsibilities

3.5. REST API surface

The API is grouped by feature area. The most relevant endpoint groups are listed below; full method signatures are available from the live Django URL configuration.

- **Authentication:** /api/register/, /api/token/, /api/token/refresh/, /api/auth/google/, /api/password_reset/, /api/password_reset/confirm/, /api/users/me/, /api/users/search/
- **Marketplace:** /api/products/ (CRUD), /api/products/{id}/favorite/
- **Laundry:** /api/machines/ (CRUD), /api/machines/{id}/report_status/
- **Chat:** /api/conversations/ (CRUD), /api/conversations/dormitory_chat/, /api/conversations/global_chat/, /api/messages/ (CRUD)
- **Events:** /api/events/ (CRUD), /api/events/{id}/rsvp/
- **Notifications:** /api/notifications/ (CRUD), /api/notifications/mark_all_as_read/
- **Profile:** /api/profile/ (GET, PATCH)

4. PROJECT MANAGEMENT WITH SCRUM

4.1. Team roles

The five members of the team filled the following Scrum and software-engineering roles:

Role	Responsibilities in DormLife
Product Owner / Scrum Master	Maintained the product backlog in Jira, prioritized features for each sprint, ran sprint planning, daily standups, sprint reviews, and retrospectives, and protected the team from scope creep.
Requirements Architect	Translated the team's high-level product ideas into concrete functional and non-functional requirements, wrote acceptance criteria, and kept the requirements documentation in sync with the implementation.
Developer (1)	Implemented features on both the front end and the back end, with primary ownership of the marketplace, exchange offers, chat, and notifications modules.
Developer (2)	Implemented features on both the front end and the back end, with primary ownership of authentication, laundry tracker, events, and the admin dashboard.
Tester	Designed the test plan and test cases, performed manual end-to-end testing, wrote automated tests for analytics and metrics, raised bug tickets in Jira, and authored the formal testing report.

Table 3. Scrum roles and responsibilities

4.2. Tools

- **Jira** (project DOSK2) for the product backlog, sprint planning, and ticket tracking.
- **GitHub** for source control, pull requests, and code review.
- **GitHub Actions** for the continuous-integration pipeline that runs the back-end test suite on every push or pull request to main.
- **Docker and Docker Compose** for reproducible staging deployments.
- **Figma** for UI mock-ups, with a dedicated design-guide file kept inside the repository.

4.3. Branching strategy

All feature work happened on dedicated branches whose name combines the Jira ticket id with a short slug, for example DOSK2-146-in-app-notifications-center. Branches were merged back into main exclusively through pull requests after code review, and the team maintained a preference for a linear history. The same convention was used for bug-fix tickets, which made it possible to trace every commit on main back to a Jira issue.

4.4. Sprint history (summary)

Across the semester, the team executed six sprints. Sprint 1 set up the repository, the basic Django and React skeletons, and the team conventions; Sprints 2 through 5 built out the feature set in priority order — authentication first, then the marketplace, chat, laundry tracker, events, and finally the in-app notification center. Sprint 6 was dedicated to hardening, formal testing, documentation, and the final

release build — no new user-facing features were added, and all 49 story points were allocated to test planning and execution, two rounds of bug fixes, CI pipeline improvements, and the preparation of this report.

5. TESTING

5.1. Testing approach

The team applied a layered testing strategy that combined automated end-to-end tests, component-level manual testing, and a continuous-integration pipeline that enforced test passage on every code change. Automated end-to-end tests were written with pytest and executed against the running application, covering all major user flows from login through to reporting. Each front-end component was also validated individually through structured manual testing, where the tester compared expected behavior against actual behavior for every interactive element, edge case, and error state. The full results of both test suites are attached to this report as separate PDF documents: the automated test report (Automated_Tests_report.pdf) and the manual testing report (Manual_testing.pdf).

5.2. Continuous integration pipeline

The project's GitHub Actions workflow was configured to trigger on every push and pull request across all branches. To avoid running unnecessary work, the pipeline incorporated the dorny/paths-filter action, which inspected the changed file tree and selectively executed only the relevant jobs:

- Backend-only modifications triggered only the Django test suite (test-backend) under Python 3.13.
- Frontend-only modifications triggered only the React compilation and test environment (test-frontend) inside the proj/ directory.
- Full-stack changes triggered both jobs sequentially.

This selective execution produced a measurable improvement in pipeline performance. On pull requests that touched only the back end, the pipeline completed in approximately 1 minute 10 seconds, compared to a baseline of 2 minutes 45 seconds when both suites ran unconditionally — a reduction of roughly 57 % in infrastructure time.

In addition to test execution, the pipeline included an automated version-increment job. A daily cron trigger scheduled at 23:00 UTC read the current build indicator from the landing page, incremented it by one, and pushed the update back to the main branch using the GitHub Actions bot with a [skip ci] flag to prevent recursive pipeline runs. This ensured that every nightly build carried a unique, monotonically increasing version number without any manual intervention.

5.3. Automated tests

The team developed a comprehensive end-to-end test suite using pytest with the pytest-html reporting plugin. The suite contained 20 test cases that exercised every major flow in the application through a real browser session. All 20 tests passed with a total execution time of 3 minutes and 1 second. The test cases covered the following areas:

- Admin login and dashboard welcome screen.
- Analytics dashboard loading and metric card rendering.
- Chat workflow including search, conversation creation, and message sending.
- Dashboard layout, admin analytics, and community buzz feed.
- Events CRUD operations and the RSVP toggle flow.

- Laundry tracker full workflow: viewing machines, updating status, admin creation and deletion.
- Marketplace grid display and item creation via the modal form.
- Marketplace item details page and the visitor contact flow.
- Notifications page lifecycle: loading, filtering, marking as read, and click-through navigation.
- Password reset request flow, including both success and error paths.
- Profile page and listings lifecycle: viewing, editing, and deleting own products.
- Registration flow with role switching and conditional field rendering for admin accounts.
- Report dashboard lifecycle: viewing pending reports and moderator actions.
- Report submission flow and cancel navigation.
- Reset password form validation and successful password change.
- Email verification for both success and failure scenarios.

The full pytest-html report with individual test durations is attached to this report as [Automated_Tests_report.pdf](#) (click on text).

In addition to the pytest suite, front-end unit tests were written with Vitest 2.0 and `@testing-library/react`. The focused Vitest suite (`Notifications.test.tsx`) covered eight scenarios: initial loading state, notification listing, unread badge styling, mark-all-as-read, filtering by status and by type, the empty state, and click-through navigation. Additional automated coverage for analytics and metrics was added during the final sprint.

5.4. Manual end-to-end test cases

Every front-end component was manually tested by the tester against a structured checklist that defined expected behavior for each interactive element. The testing covered fifteen components in total: Analytics Dashboard, Chat, Dashboard, Events, Item Details, Laundry, Login, Marketplace, Notifications, Profile, Register, Report Page, Report Dashboard, Reset Password, and Verify Email. For each component, the tester documented what was expected to see and what was actually observed, confirming correct behavior in areas such as:

- Loading and empty states rendering correctly across all pages.
- Conditional rendering based on user role (student, admin, doorkeeper).
- Form validation, submission, and error handling for every input form in the application.
- Optimistic UI updates in chat message sending and notification mark-as-read.
- Real-time polling behavior in chat, laundry timers, and notification refresh.
- CRUD operations for marketplace listings, events, laundry machines, and reports.
- Navigation logic, including route guards, redirect after login, and click-through from notifications to the correct target page.
- Responsive layout and hover interactions on both desktop and mobile viewports.
- Image handling with fallback rendering for missing profile photos and product images.

All tested behaviors matched their expected outcomes. The complete manual testing report with per-component breakdowns is attached to this report as [Manual_testing.pdf](#). (click on text).

5.5. Bugs found and resolved

Manual testing during Sprint 6 surfaced a number of bugs, all of which were resolved across two Jira tickets — DOSK2-236 (first round) and DOSK2-250 (second round) — before the final build:

- Exchange-offer notifications were not being created because the signal handler referenced a non-existent attribute on the ExchangeOffer model. The handler was corrected to use `target_product.seller` as the recipient.
- Global-chat creation occasionally produced duplicate conversations because the create and accessor paths used different casings of the type value. Both paths were normalized to a single consistent value.
- Clicking a single notification incorrectly invoked the mark-all-as-read endpoint instead of patching the individual record. The click handler was updated to issue a PATCH against the individual notification.
- Several pages did not handle a 401 response from an expired access token gracefully. An Axios response interceptor was added to call the token-refresh endpoint automatically before retrying the failed request.

6. CONCLUSION

Across one semester, the team has delivered DormLife — a full-stack web application designed for students who live in dormitories — using the Agile and Scrum practices and the tooling that the Software Engineering course is built around. All core features identified at the start of the project (marketplace with buy / sell / exchange, laundry tracker, multi-level chat, events, profile, and an in-app notification center) are implemented and connected end-to-end. The system is built on a modern, industry-relevant stack of React, TypeScript, Django REST Framework, and PostgreSQL, runs in a reproducible Docker-based staging environment, and is supported by a GitHub Actions continuous-integration pipeline.

Beyond the product itself, the project gave the team direct experience with the day-to-day mechanics of professional software development: writing user stories with acceptance criteria, breaking work into ticketed sprint backlogs in Jira, branching per ticket, opening pull requests, reviewing each other's code, and resolving bugs surfaced by a dedicated tester rather than the developers themselves. The Sprint 6 backlog shown in this report demonstrates that even in the final iteration the team continued to allocate explicit capacity to testing, infrastructure, and documentation, rather than only adding features.

There are clear opportunities for future work. Polling-based chat could be replaced by WebSocket-based real-time messaging via Django Channels; an Axios response interceptor would handle silent token refresh; client-side route guards would harden the front end against direct URL access without a token; and end-to-end test automation could be expanded beyond notifications to cover the marketplace and chat. These items have been recorded in the project backlog and are scoped as logical follow-up sprints. As it stands at the end of the course, DormLife meets the aim set out in section 1.2 and constitutes a complete and demonstrable deliverable.

REFERENCES

- [1] Schwaber, K., Sutherland, J. (2020). The Scrum Guide — The Definitive Guide to Scrum: The Rules of the Game. <https://scrumguides.org/scrum-guide.html>
- [2] Atlassian. Jira Software documentation. <https://www.atlassian.com/software/jira/guides>
- [3] Chacon, S., Straub, B. (2014). Pro Git, 2nd edition. Apress. <https://git-scm.com/book>
- [4] React Team. React 18 documentation. <https://react.dev/>
- [5] Microsoft. TypeScript documentation. <https://www.typescriptlang.org/docs/>
- [6] Vite. Vite — Next Generation Frontend Tooling. <https://vitejs.dev/>
- [7] Tailwind Labs. Tailwind CSS documentation. <https://tailwindcss.com/docs>
- [8] shadcn. shadcn/ui component library. <https://ui.shadcn.com/>
- [9] Django Software Foundation. Django documentation. <https://docs.djangoproject.com/>
- [10] Encode OSS. Django REST Framework. <https://www.django-rest-framework.org/>
- [11] django-rest-framework-simplejwt. Simple JWT documentation. <https://django-rest-framework-simplejwt.readthedocs.io/>
- [12] Google. Google Identity — OAuth 2.0 documentation. <https://developers.google.com/identity/protocols/oauth2>
- [13] PostgreSQL Global Development Group. PostgreSQL 15 documentation. <https://www.postgresql.org/docs/15/>
- [14] Docker Inc. Docker Compose documentation. <https://docs.docker.com/compose/>
- [15] GitHub. GitHub Actions documentation. <https://docs.github.com/en/actions>
- [16] Vitest. Vitest — A Vite-native testing framework. <https://vitest.dev/>
- [17] Testing Library. React Testing Library documentation. <https://testing-library.com/docs/react-testing-library/intro/>